

Lu Delivery – Peru: Sistema de Pedidos para Restaurantes com Programação Orientada a Objetos

Franklin Ferreira dos Santos, Giovanna Salomão Rodrigues, Kailla de Souza Costa, Lucas de Jesus Barreto, Lucas Silva Oliveira

Sistemas de Informação – Centro Universitário de Excelência –
(UNEX) – Feira de Santana, BA - Brasil

franklinferreira280@gmail.com, irodrigues708@gmail.com,
kaillacsouza@gmail.com, ljbarreto04@gmail.com, lucasolv.info@gmail.com

Abstract. *This paper details the refactoring process of the Lu Delivery - Peru ordering system. The initial version, while functional, had high coupling between its classes, making maintenance and extension difficult. The refactoring focused on applying Design Patterns, particularly Observer, to create a more robust, decoupled, and extensible system. The result is an architecture where business classes (such as Customer and DeliveryQueue) can dynamically react to order state changes without the Order class needing to directly understand them, consolidating the principles of cohesion and loose coupling of Object-Oriented Programming.*

Resumo. *Este trabalho detalha o processo de refatoração do sistema de pedidos Lu Delivery - Peru. A versão inicial, embora funcional, apresentava um alto acoplamento entre suas classes, dificultando a manutenção e extensão. A refatoração focou na aplicação de Padrões de Projeto, com destaque para o **Observer**, para criar um sistema mais robusto, desacoplado e extensível. O resultado é uma arquitetura onde as classes de negócio (como Cliente e FilaEntrega) podem reagir dinamicamente a mudanças de estado dos pedidos sem que a classe Pedido precise conhecê-las diretamente, consolidando os princípios de coesão e baixo acoplamento da Programação Orientada a Objetos.*

1. Introdução

A primeira versão do sistema *Lu Delivery - Peru* cumpriu os requisitos funcionais básicos de um sistema de pedidos. No entanto, a análise do projeto inicial revelou oportunidades significativas de melhoria em sua arquitetura, especialmente no que tange à comunicação entre os objetos.

O objetivo principal desta refatoração foi evoluir a arquitetura do sistema para aderir mais estritamente aos princípios de design da Orientação a Objetos, como o baixo acoplamento e a alta coesão. Para alcançar tal objetivo, foi aplicado o padrão de projeto **Observer**, que permite que múltiplos objetos sejam notificados sobre mudanças de estado de um objeto central, de forma dinâmica e desacoplada.

Este relatório apresentará a fundamentação teórica por trás dos padrões de projeto utilizados, detalhará a metodologia de refatoração aplicada, e exibirá

os resultados obtidos, incluindo o novo diagrama de classes e exemplos práticos do sistema em funcionamento.

2. Fundamentação Teórica

Para compreender as decisões tomadas na refatoração, é crucial revisitar alguns conceitos teóricos fundamentais.

2.1 Relações entre Objetos: Composição

Na Programação Orientada a Objetos, a **Composição** é uma forma forte da relação de associação, que representa um relacionamento "todo-parte". Nela, o ciclo de vida do objeto-parte está atrelado ao do objeto-todo. Se o "todo" é destruído, a "parte" também é. Em nosso sistema, a relação entre Pedido e ItemPedido é uma composição: um ItemPedido não faz sentido e não pode existir fora do contexto de um Pedido.

2.2 Padrão de Projeto Observer

O **Observer** é um padrão de projeto comportamental que define uma dependência um-para-muitos entre objetos. Quando o estado de um objeto (o "Subject" ou "Observado") muda, todos os seus dependentes (os "Observers" ou "Observadores") são notificados e atualizados automaticamente.

Subject (Observado): É o objeto de interesse. Ele mantém uma lista de seus observadores e fornece uma interface para adicionar e remover observadores. No nosso projeto, a classe Order (Pedido) atua como o Subject.

Observer (Observador): É a interface que todos os observadores concretos devem implementar. Ela define um método de atualização, que é chamado quando o Subject muda de estado. Em nosso projeto, esta é a interface OrderObserver.

Concrete Observer (Observador Concreto): São as classes que implementam a interface Observer e reagem às notificações. Em nosso projeto, Customer (Cliente) e a nova classe FilaEntrega são observadores concretos.

O principal benefício deste padrão é o **baixo acoplamento**. O Subject não precisa saber nada sobre seus observadores, exceto que eles implementam a interface Observer. Isso torna o sistema extremamente flexível, permitindo que novos observadores sejam adicionados sem modificar o código do Subject.

3. Metodologia e Refatoração

A refatoração foi guiada pela necessidade de desacoplar a lógica de notificação da classe Order. Na versão anterior, se quiséssemos, por exemplo, enviar um e-mail ao cliente quando o pedido fosse aceito, teríamos que adicionar essa lógica diretamente dentro da classe Order, violando o Princípio da Responsabilidade Única.

O processo de refatoração seguiu os seguintes passos:

1. **Criação da Interface OrderObserver:** Foi definida a interface

OrderObserver com um único método: `onStatusChanged(Order order, OrderStatus previous, OrderStatus current)`. Este método serve como o contrato que todos os observadores devem seguir.

2. Modificação da Classe Order (Subject):

A classe Order foi modificada para manter uma lista de OrderObserver.

Foram criados métodos para gerenciar essa lista: `addObserver()` e `removeObserver()`.

O método `updateStatus()` foi implementado para, além de alterar o status, iterar sobre a lista de observadores e invocar o método `onStatusChanged()` de cada um, notificando-os da mudança.

3. Implementação dos Observadores Concretos:

A classe Customer passou a implementar OrderObserver. Agora, um cliente pode "ouvir" as atualizações de seus próprios pedidos e reagir a elas, como imprimir uma mensagem de confirmação.

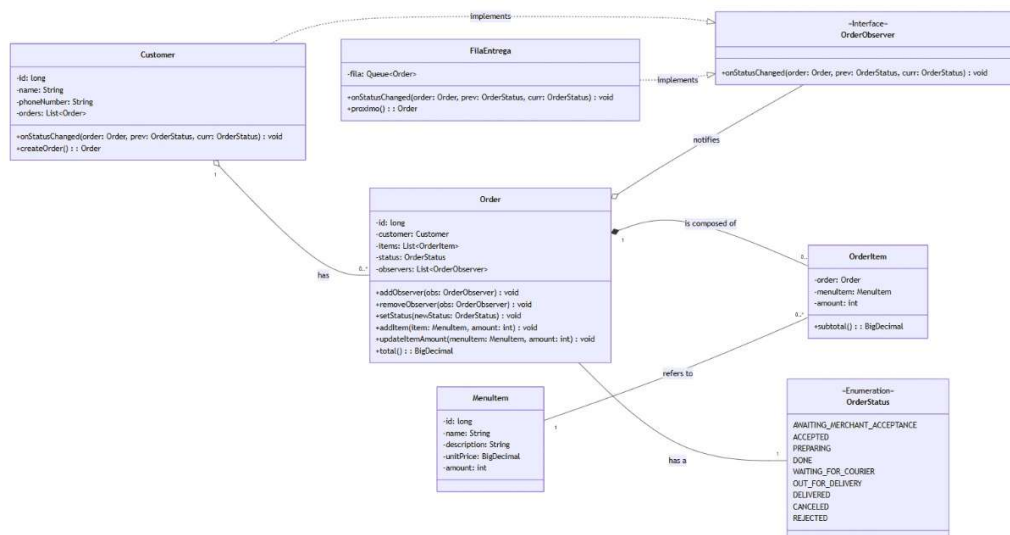
Foi criada a classe FilaEntrega, que também implementa OrderObserver. Sua responsabilidade é observar os pedidos e, quando um pedido atinge o status `WAITING_FOR_COURIER`, adicioná-lo a uma fila de entrega. Isso separa completamente a lógica de entrega da lógica do pedido.

4. Resultados e Discussões

A aplicação do padrão Observer transformou a arquitetura do sistema, tornando-a mais modular e robusta.

4.1 Novo Diagrama de Classes

O diagrama abaixo ilustra a nova estrutura do sistema. Note a relação entre Order (Subject) e a interface OrderObserver, e como Customer e FilaEntrega implementam essa interface, caracterizando o padrão.



Descrição do Diagrama:

Order agora possui uma associação com **OrderObserver**, indicando que pode ter múltiplos observadores.

Customer e **FilaEntrega** implementam a interface **OrderObserver** (indicado pela seta de realização pontilhada).

A relação entre **Order** e **OrderItem** permanece como Composição (indicada pelo losango preenchido), pois um **OrderItem** depende existencialmente de um **Order**.

5. Conclusão

A refatoração do sistema *Lu Delivery - Peru* demonstrou o poder dos padrões de projeto na criação de software de alta qualidade. A transição de uma arquitetura acoplada para um modelo desacoplado com o uso do padrão **Observer** não apenas resolveu as questões apontadas na análise inicial, mas também preparou o sistema para futuras expansões. Agora, adicionar novas funcionalidades — como um notificador por SMS ou um painel de monitoramento em tempo real — é uma tarefa trivial que consiste em criar um novo observador, sem a necessidade de alterar as classes de domínio existentes.

Este projeto reforçou o entendimento de que a qualidade de um software não reside apenas em sua funcionalidade, mas também em sua estrutura, manutenibilidade e capacidade de evoluir.

Referências

Guanabara, G. (2020). Curso de Java POO. [s.l.]: Curso em Vídeo. Disponível em: https://www.youtube.com/watch?v=KIIL63MeyMY&list=PLHz_AreHm4dkqe2aR0tQK74m8SFe-aGsY. Acesso em: 19 ago. 2025.

DEVMEDIA. (2010). Padrão de Projeto Singleton em Java. [s.l.]: DevMedia. Disponível em: <https://www.devmedia.com.br/padrao-de-projeto-singleton-em-java/26392>. Acesso em: 22 ago. 2025.

DEVMEDIA. Encapsulamento em Java. São Paulo: DevMedia, 2018. Disponível em: <https://www.devmedia.com.br/encapsulamento-em-java/29106>. Acesso em: 23 ago. 2025.